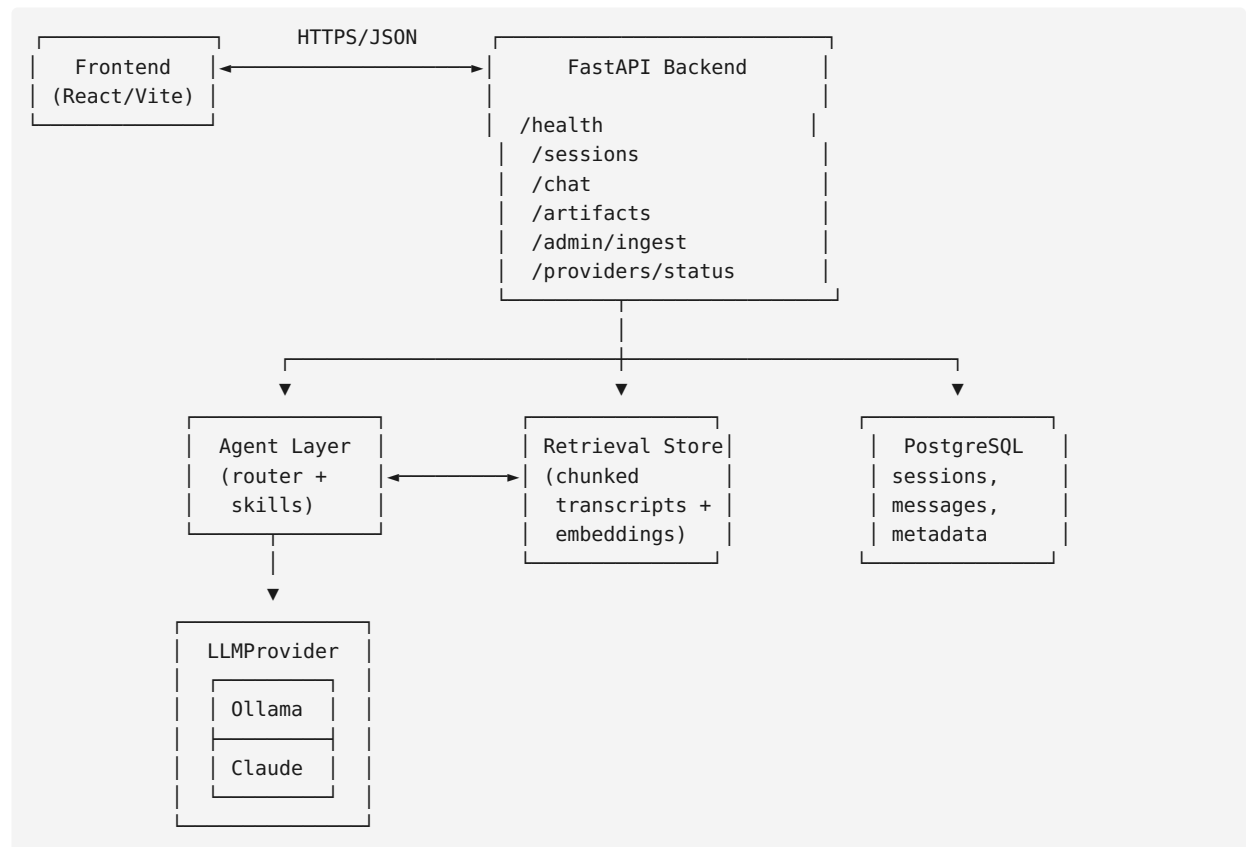


architecture

Architecture — The Lenny Growth Assistant

Status: designed at Stage A, ahead of implementation. This is the target architecture; the requirements checklist tracks build status per component.

1. System Overview



2. Component Boundaries

- **Frontend**: chat UI, session list, Artifact Viewer, provider indicator. Talks only to the FastAPI backend over JSON — no direct DB or LLM access.
- **Backend (FastAPI)**: owns all business logic, validation, persistence, and orchestration. Stateless per-request except for DB-backed session state.
- **Agent layer**: intent router + skill modules (`grounded_qa`, `ship30_essay`, `artifact_generation`). Each skill is a separate module with a defined input/output contract — not one shared prompt.
- **Retrieval store**: transcript chunks + embeddings, queried by skills, never called directly by the frontend.
- **LLMProvider abstraction**: single interface (`generate(messages, **kwargs) -> Response`) implemented by `OllamaProvider` and `ClaudeProvider`; selected via `MODEL_PROVIDER` env var at process start, no code change required to switch.
- **PostgreSQL**: sessions, messages, timestamps, lightweight anonymous client metadata.

3. Database Schema (target)

```
CREATE TABLE sessions (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```

    client_id TEXT NOT NULL,           -- anonymous per-browser identifier
    created_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
    title TEXT                         -- derived from first message, nullable
);

CREATE TABLE messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    role TEXT NOT NULL CHECK (role IN ('user','assistant','system')),
    content TEXT NOT NULL,
    sources JSONB,                     -- array of {episode, timestamp/section, url} when grounded
    provider TEXT,                     -- 'claude' | 'ollama', which one produced this message
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE artifacts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    session_id UUID NOT NULL REFERENCES sessions(id) ON DELETE CASCADE,
    message_id UUID REFERENCES messages(id) ON DELETE SET NULL,
    kind TEXT NOT NULL CHECK (kind IN ('markdown','html')),
    content TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

Indexes: `messages(session_id, created_at)`, `sessions(client_id)`.

4. API Endpoints (target)

Method	Path	Purpose	Notes
GET	/health	Liveness + component status (DB, provider reachability)	200 always if process is up; body reports sub-component health
POST	/sessions	Create a new session	Returns session id
GET	/sessions	List sessions for client	For history sidebar
GET	/sessions/{id}/messages	Fetch message history	
POST	/chat	Send a message, get a grounded/agent response	Body: {session_id, message} ; Response: {message, sources[], provider}
POST	/artifacts	Generate a Markdown/HTML artifact from conversation	Body: {session_id, kind, instructions}
GET	/providers/status	Current provider, availability of each configured provider	Backs the UI indicator
POST	/admin/ingest	Re-run transcript ingestion/indexing	Protected/admin-only in practice; documented, not exposed publicly in UI

All endpoints use Pydantic request/response models; validation errors return structured 422 bodies ({detail: [...]}); unexpected errors return structured 5xx bodies with a generic message ({error: "..."}) — no stack traces to the client, full trace to logs.

5. Ingestion / Retrieval Flow (RAG)

1. **Ingestion:** `ingestion/load.py` reads transcript files from `transcripts/` (populated by cloning the ChatPRD repo locally — see README).
2. **Chunking:** split per-episode transcripts into ~500–800 token overlapping chunks, preserving episode title/URL/approximate section as metadata.
3. **Indexing:** embed each chunk (via the active provider’s embedding capability, or a local sentence-embedding model as fallback) and store in a local vector index (e.g., a simple FAISS or SQLite-based store) alongside chunk metadata.
4. **Retrieval:** on each chat request, embed the user’s query, retrieve top-k chunks by similarity, pass them into the agent’s grounding context with explicit source metadata.

5. **Source tracing:** every retrieved chunk carries `{episode_title, source_url, chunk_id}`; the response's `sources[]` field is populated directly from what was actually retrieved — never fabricated post-hoc.
6. **Refresh strategy:** re-run ingestion via `POST /admin/ingest` or `scripts/ingest.sh` whenever the transcript repo is updated; index is rebuilt idempotently (existing chunks keyed by content hash to avoid duplicate re-embedding).

6. Agent Routing

A lightweight intent classifier (rule-based first pass + LLM fallback for ambiguous cases) routes each `/chat` request to one of: - `grounded_qa` skill — default path for questions. - `ship30_essay` skill — triggered by explicit essay/content requests. - `artifact_generation` skill — triggered by explicit “generate a doc/HTML” requests, or invoked as a follow-up action on a prior grounded answer.

Each skill receives the conversation history + retrieved context and returns a typed result (`text` , `sources` , optional `artifact`).

7. Model Provider Abstraction & Toggle

```
class LLMPProvider(ABC):
    def generate(self, messages: list[Message], **kwargs) -> ProviderResponse: ...
    def is_available(self) -> bool: ...

class OllamaProvider(LLMPProvider): ...
class ClaudeProvider(LLMPProvider): ...
```

- Selected via `MODEL_PROVIDER=ollama|claude` env var — no code change to switch.
- `/providers/status` and the UI badge reflect the active provider and its live availability (`is_available()` ping).
- **Fallback behavior:** if the configured provider is unavailable at request time (e.g., Ollama not running), the backend returns a structured error indicating the provider is down rather than silently switching providers — silent fallback would undermine the “provider must be visible” requirement. Automatic fallback to a secondary provider is configurable via `PROVIDER_FALLBACK=true`, off by default, documented in `.env.example`.

8. Artifact Generation & Rendering Security

- Skills generate Markdown or HTML/CSS as plain text, stored in the `artifacts` table, returned to the frontend.
- **Frontend rendering:**
 - Markdown → rendered through a Markdown renderer with HTML disabled (no raw HTML pass-through).
 - HTML/CSS → rendered inside a sandboxed `<iframe srcdoc=“...”>`. The `sandbox` attribute is set **without** `allow-scripts` and **without** `allow-same-origin` together — preventing script execution and preventing the iframe from accessing the parent page’s storage/cookies.
- **What’s permitted:** structural HTML + inline CSS for layout/typography artifacts (one-pagers, styled documents).
- **What’s blocked:** `<script>` execution, external resource loading beyond same safe defaults, access to parent window/DOM, form submission to arbitrary endpoints.
- **Why this is appropriate:** the assignment explicitly says treat generated HTML as untrusted; iframe sandboxing without script/same-origin permissions is the standard low-complexity mitigation that still allows rich visual output.
- **Remaining risk:** CSS-based exfiltration/phishing-style visual tricks are still technically possible within a sandboxed iframe (no code execution, but layout can still mislead) — documented as an accepted residual risk, not eliminated.

9. Deployment Topology

- `docker-compose.yml` brings up: `backend` (FastAPI), `frontend` (static build served or dev server),

postgres .

- Ollama runs **on the host**, not in Compose, since GPU/performance characteristics vary by machine — backend connects via `OLLAMA_HOST` env var (default `http://host.docker.internal:11434`).
- Cloud Claude access via `ANTHROPIC_API_KEY` env var, never committed.

10. Failure Scenarios & Recovery

Failure	Behavior
Missing API key	Backend boots; <code>/providers/status</code> reports cloud provider unavailable; chat requests routed to it return a structured “provider not configured” error, not a crash
Ollama unreachable	<code>is_available()</code> fails fast (short timeout); user sees “local model unavailable” message
Model timeout	Request-level timeout (e.g., 30s) wraps provider calls; times out to a structured error, not a hang
Empty retrieval result	Agent explicitly responds that the knowledge base doesn’t cover the question
DB connection failure	Connection pool errors are caught; <code>/health</code> reports DB down; chat can still respond (degraded: no persistence) rather than 500ing the whole app — documented as best-effort degradation
Invalid user input	Pydantic validation → 422 with field-level detail
Artifact generation failure	Returns structured error; chat conversation itself is unaffected
Unsafe artifact content	Caught by sandboxing at render time regardless of generation-time filtering (defense in depth)

11. Observability

- Structured JSON logs (via Python `logging` + a simple formatter) tagged by component (`provider` , `retrieval` , `db` , `artifact` , `api`).
- Never log: API keys, full user message content at DEBUG-suppressed levels in production config (documented), secrets.
- Each failure scenario in §10 has a corresponding log line at `WARNING` / `ERROR` with enough context (session id, provider, error type) to diagnose without exposing secrets.